ostree

Helix OTA — Stack Research: OSTree / libostree / rpm-ostree

Field	Value
Revision	1
Created	2026-06-07
Last modified	2026-06-07
Status	active
Status summary	Evidence-based research note on libostree (OSTree) and rpm-ostree as a candidate engine for **file-based atomic deployments** in the Helix OTA **Linux phase** (all flavors). Covers the deployment model, pinning/rollback, static deltas, the HTTP server side, licensing/maturity, and Android (AOSP) fit. OSTree is explicitly **not** an A/B partition scheme; it is a parallel deployment model on a single rootfs partition — relevant to Linux, largely orthogonal to the Android-15-first native A/B path.
Issues	A few quantitative/operational claims could not be pinned to a single canonical figure (exact storage-overhead formula, precise GC retention defaults, delta size ratios). These are tagged “UNVERIFIED — needs confirmation” inline and must be benchmarked before any ADR commits to OSTree.
Issues summary	No number is invented; unverifiable claims are flagged, not guessed (HelixConstitution §11.4.6).
Fixed	initial research note
Fixed summary	Authored from the official ostreedev/ coreos docs and man pages consulted live on 2026-06-07; all URLs cited are

Field	Value
	pages actually fetched or returned by search this session.
Continuation	Feed into the Linux-phase OS-adapter spec and the engine-selection ADR(s). If OSTree advances past screening, prototype an `archive` repo + static-delta server and measure delta sizes + GC behavior on real images; cross-link with the Mender/hawkBit landscape report. Re-run web verification before any production decision (releases move on a date-based cadence).

Table of contents

§1. Scope & relevance to Helix

Helix OTA is **Android 15 first** (Orange Pi 5 Max), using AOSP-native A/B (Virtual A/B + compression) with a custom Go control plane, and later extends to **Linux (all flavors)**, Windows, and other OSes via pluggable OS adapters (per the master design spec, §1). OSTree is examined here as a candidate **Linux-phase** file-based atomic-deployment engine — i.e., the OS-adapter backend for Linux targets, not for the Android device path. Per the request, the focus is: file-based atomic deployments, pinning/rollback, static deltas, the server side, relevance to the Linux phase, and the fact that it is **not A/B**.

Key framing up front: **OSTree is not an A/B partitioning system.** It achieves atomicity and rollback through a content-addressed object store and multiple parallel "deployments" hardlinked onto a **single** root partition, swapped via the bootloader. This is a fundamentally different mechanism from Android's dual-slot A/B, which is why OSTree is interesting for Linux but does not displace the locked Android A/B strategy.

§2. What OSTree is (mechanism)

libostree (formerly "OSTree") is a library + CLI for **versioned, atomic, bootable filesystem trees**, described upstream as "git for operating system binaries." It is a userspace tool that "will work on top of any Linux filesystem" — i.e., it is Linux-oriented by design (ostreedev introduction; GitHub README).

The repository is a **content-addressed object store** using **SHA-256** checksums, with these object types (repo format docs):

- **Commit object** — metadata (timestamp, log message) plus a reference to a dirtree/dirmeta checksum pair describing the root directory.
- **Dirtree object** — sorted arrays of `(filename, content checksum)` and `(filename, dirtree checksum, dirmeta checksum)` for subdirectories.
- **Dirmeta object** — directory metadata (uid/gid/mode/xattrs), split out to avoid duplicating extended-attribute listings.
- **Content object** — file metadata (uid, gid, mode, symlink target, xattrs) followed by file content for regular files.

Notably, "the OSTree data format intentionally does not contain timestamps," which makes builds reproducible/syncable across distributed builders (repo format docs).

§3. File-based atomic deployments

A **deployment** is a specific checksummed commit, checked out as a set of **hardlinks into the object store**. Because files are content-addressed and deduplicated, multiple deployments coexist on one partition and share identical objects; an upgrade "only uses disk space proportional to the new files, plus some constant overhead" (atomic-upgrades docs / search summary). The exact "constant overhead" figure is **UNVERIFIED — needs confirmation** (benchmark required).

Atomicity guarantee, quoted from upstream: with `ostree admin upgrade`, "if the system crashes or you pull the power, you will have either the old system, or the new one." Mechanism details (atomic-upgrades docs):

- The OS tree is read-only (`/usr`); `/etc` is handled via a **3-way merge** between the current `/etc`, the old default config, and the new deployment's `/usr/etc`, preserving local edits (inspect with `ostree admin config-diff`). `/var` is shared/persistent.
- Boot config swap uses a **"swapped directory" pattern**: `/boot` (or `/ostree/boot.N`) is populated in the inactive copy, then the symlink is atomically flipped. Bootloader entries are written under `/boot/loader/entries/` with a kernel argument of the form `ostree=/ostree/deploy/$stateroot/deploy/$checksum` so initramfs mounts the chosen deployment as root.
- Objects are downloaded individually (HTTP model), each validated against its checksum before being stored in `/ostree/repo/objects/`.

Net: atomicity is achieved at the **filesystem-tree** level via hardlink checkouts + an atomic bootloader pointer swap — no second partition required.

§4. Pinning & rollback

Rollback model. OSTree keeps the previous deployment available automatically. After one upgrade you have a booted deployment plus a rollback deployment; on the next upgrade the old rollback is discarded and the previously booted one becomes the new rollback (search summary of apertis/ostree docs). Rollback is performed by **reordering the deployment list** — historically via `ostree admin deploy` / status manipulation; note there is an open upstream request (`ostreedev/ostree#1808`) about a dedicated `ostree admin rollback`, and in practice rpm-ostree provides `rpm-ostree rollback`. The exact current `ostree admin` rollback invocation is **UNVERIFIED — needs confirmation** (verify against the installed version's man pages before relying on it).

Pinning (`ostree admin pin`, man page consulted):

- Synopsis: `ostree admin pin {INDEX}`.
- `INDEX` is a non-negative integer **or** one of the aliases `booted`, `pending`, `rollback`.
- A pinned deployment "will not be garbage collected by default."
- `--unpin` / `-u` undoes a pin.
- Pinned deployments are exempt from `ostree admin cleanup` GC; unpinned ones become eligible for collection. Status output (`ostree admin status` / `rpm-ostree status`) shows `Pinned: yes`.

Retention / GC. By default OSTree retains a small number of deployments (booted + rollback) and garbage-collects the rest on cleanup; pinning is the mechanism to retain a known-good baseline across rebases. The exact default number of retained deployments / GC depth is **UNVERIFIED — needs confirmation** (it is configurable and distro-dependent; confirm via `ostree admin cleanup` docs + config for the target image).

For Helix this is attractive: pinning gives an explicit "known-good" anchor the control plane can guarantee survives, complementing (not replacing) the automatic last-good rollback.

§5. Static deltas

Default OSTree HTTP updates fetch **one object per changed file**, which is inefficient for large infrequent updates. **Static deltas** are pre-computed, server-generated diffs between two specific commits, trading "server-side storage (and one-time compute cost)" for client network-bandwidth efficiency (formats docs).

Details (formats docs):

- Deltas are generated between arbitrary commits; a special **"from NULL"** mode produces a full initial download (useful for first install / container scenarios).
- On-disk layout: `deltas/$fromprefix/$fromsuffix-$to`, each delta being a directory with a **superblock** (metadata, target commit, fallback info) plus integer-named **pack files**.
- Delta payloads are described as "restricted programs" — an "updates as code" model (inspired by ChromiumOS) executed by the client during a compilation phase.

- Compression strategies: **bsdiff** for similarly named/sized files (effective on executables); bytecode instructions to reuse common data sections; and **fallback objects** for large incompressible files (e.g., compressed initramfs) fetched as normal objects.
- The repository **summary file** lists all static deltas with their metadata checksums, alongside ref info, so clients can discover them.

Generation is via `ostree static-delta generate` (CLI), and deltas are served as static files. Concrete delta-size ratios vs full pulls are workload-dependent and **UNVERIFIED — needs confirmation** (must be measured on representative Helix Linux images).

§6. Server side

The server side is deliberately simple: an **`archive`-mode repository served over plain HTTP** (repo format / formats docs). Properties:

- `archive` mode stores content objects gzip-compressed with uncompressed metadata, "designed for serving via plain HTTP" — i.e., any static web server / CDN / object store works; no special server daemon is required for the base pull model.
- Incremental: clients fetch only changed objects (or a static delta) referenced from the commit/summary.
- Static deltas (see §5) are pre-generated server-side and dropped into the repo for bandwidth efficiency.
- Local on-device repos use **bare** / **bare-user** modes (uncompressed, hardlink farm; `bare-user` drops uid/gid for non-root/cross-env use); plus `bare-split-xattrs` and `bare-user-only` variants for xattr-constrained filesystems.

Implication for Helix's Go control plane: OSTree's "server" is essentially **static artifact hosting + a commit/summary/delta generation pipeline**. The Go control plane would not need to reimplement the transport; it would orchestrate `ostree commit` / `ostree static-delta generate` / `ostree summary -u` on build, publish the `archive` repo to S3/MinIO (already in the Helix stack), and layer Helix's own rollout/telemetry/auth on top. This is a clean wrap boundary (CLI + static files), not a tightly coupled server protocol.

§7. rpm-ostree & bootc

rpm-ostree is a hybrid image/package system: libostree as the base image format plus RPM (sharing libdnf with dnf) on client and server (coreos.github.io/rpm-ostree; GitHub README). It composes RPMs server-side into an OSTree commit that clients replicate bit-for-bit with fast incremental updates, and supports **package layering** (client-side overlay of extra RPMs as "OS extensions"). Rollback affects `/usr` but not `/etc` or `/var`.

It also supports **container-native OSTree** (via ostree-rs-ext): OCI/Docker images as a transport for bootable OSes. Per upstream, **active feature development has shifted to `bootc`, dnf, and the bootable-container ecosystem**, while rpm-ostree "continues to be supported" and is widely used (Fedora CoreOS/Silverblue, RHEL for Edge). Red Hat documents a **migration path from rpm-ostree to bootc** (RHEL 10 docs). For Helix this is a strategic signal: **OSTree (the core library) remains the**

durable, supported substrate, but the higher-level RPM-centric tooling is in a managed transition toward bootc — a forward-looking Linux adapter should evaluate bootc alongside raw libostree, not bet on rpm-ostree as a growth area.

§8. OSTree vs A/B (and why it matters for Helix)

This is the crux for Helix. OSTree is **not** an A/B partition scheme:

Dimension	Android native A/B (Helix locked)	OSTree
Unit	Two whole partitions (slot A / slot B)	Multiple deployments (hardlink checkouts) on one rootfs partition
Atomicity	Slot switch in bootloader	Atomic bootloader symlink/pointer swap to a checksummed deployment
Storage cost	~2× partition for the OS	Deduplicated; only delta of changed objects + constant overhead (UNVERIFIED exact figure)
Granularity	Block/partition image (`payload.bin`)	File/object level, content-addressed
Rollback	Slot revert + boot-failure auto-rollback	Reorder deployment list; auto-keeps previous deployment
Layering	None (image swap)	Package/config layering (rpm-ostree)
Native platform	AOSP / `update_engine`	Linux userspace (any FS)

OSTree's file-level dedup can be far more storage-efficient than 2× A/B and supports many retained versions cheaply (good for pinning multiple known-good baselines). But it does **not** provide A/B's independent-partition fault isolation in the same way; its safety comes from atomic pointer swap + read-only `/usr` + automatic rollback deployment. For Helix, OSTree is a **Linux-phase option**, not a competitor to the Android A/B decision.

§9. Android fit

Two distinct things must not be conflated:

1. **OSTree's Android *bootloader* backend (`aboot-deploy`).** OSTree has an Android-bootloader backend that reads `androidboot.slot_suffix=`, writes to the alternate `boot_a`/`boot_b` slot, and sets `/ostree/root.a` or `/ostree/root.b` (bootloaders docs; search summary). This targets devices that *use the Android boot image format / aboot* (notably automotive Linux images, e.g., CentOS Automotive SIG), running **Linux userspace** under that bootloader. It is **not** OSTree managing the AOSP Android OS itself.

2. **OSTree on actual AOSP/Android 15.** OSTree expects a Linux rootfs it can lay out as `/ostree/...` with read-only `/usr`, a 3-way-merged `/etc`, and bootloader entries. AOSP's runtime (system/vendor/product partitions, `update_engine`, Virtual A/B with dm-snapshot compression, dm-verity, SELinux policy, dynamic partitions) does **not** match this model. Using OSTree to deploy the Android OS would be a non-idiomatic, high-risk port. **No evidence was found of OSTree being used to manage a standard AOSP system image;** AOSP's own A/B (`update_engine`) is the native, supported path — and is exactly what Helix has locked for the Android phase.

Conclusion: OSTree is a poor fit for the Android-15-first device path and a good candidate only for the later Linux phase. Treat the "Android bootloader backend" as a Linux-on-aboot feature, not Android OS support.

§10. Go control-plane fit & wrapability

- **Wrap boundary is clean.** The server side is `ostree` CLI commands (`commit`, `static-delta generate`, `summary -u`) producing an `archive` repo of static files. A Go control plane shells out (or uses libostree via cgo/GObject Introspection — heavier) to build/publish, then serves the repo from S3/MinIO behind Helix's existing transport (Brotli/HTTP3→HTTP2). No bespoke server daemon to integrate.
- **Client side** is the `ostree` binary + libostree on the Linux device; Helix's Linux OS-adapter would invoke `ostree admin upgrade` / `deploy` / `pin` / `cleanup` and parse `ostree admin status` (machine-readable output / `--json` availability is **UNVERIFIED — needs confirmation** for the target version).
- **No native Go binding of record.** A Rust crate exists (`ostree` on docs.rs, e.g. 0.20.x — GObject-Introspection bindings); for Go the pragmatic path is CLI orchestration. A native Go binding maturity claim would be **UNVERIFIED**; do not assume one.
- Pinning maps neatly onto Helix's "guaranteed known-good baseline" requirement; rollback maps onto the zero-corruption guarantee for Linux targets.

§11. License & maturity

- **License: LGPL-2.1-or-later** (upstream states "Currently, that's LGPLv2+", canonical in the `COPYING` file). LGPL is compatible with wrapping via CLI and with linking; redistribution of modified libostree must respect LGPL terms. Confirm the exact SPDX string in `COPYING` at the pinned version before shipping. (rpm-ostree is separately licensed — verify its `COPYING`; **UNVERIFIED** here.)
- **Maturity:** Mature and production-proven. Date-based releases; **latest observed release v2025.7** (GitHub releases; release date reported as "November 11" by search — year/exact date **UNVERIFIED**, confirm on the releases page). Releases are GPG-signed git tags. Deployed at scale in Fedora Silverblue/CoreOS, RHEL for Edge, endless OS, Toradex Torizon, Apertis, and the CentOS Automotive SIG.
- **Governance:** Active upstream (`ostreedev/ostree`); the broader ecosystem momentum is moving toward **bootc** for bootable-container workflows, with libostree as the stable substrate.

- Star count / contributor count: **not collected this session — do not cite a number** (would be fabrication).

§12. Pros / cons for Helix

Pros - True file-based atomic deployments with automatic rollback; matches the zero-corruption guarantee for Linux targets. - Storage-efficient via content-addressed dedup — many retained/pinned versions at low cost (vs ~2× for A/B). - Explicit **pinning** of known-good deployments, exempt from GC — direct fit for a control-plane-guaranteed baseline. - **Static deltas** give bandwidth-efficient updates without a custom diff format. - **Trivial server side**: `archive` repo over plain HTTP/static hosting/CDN/S3 — slots straight into Helix's MinIO/S3 + Go control plane with a clean CLI wrap boundary. - Mature, widely deployed, LGPL, GPG-signed releases.

Cons - **Not A/B** and **not an Android-OS solution** — irrelevant to the Android-15-first device path; value is Linux-phase only. - Linux-only by design (userspace on a Linux FS); the "Android bootloader" backend is Linux-on-aboot, not AOSP. - No first-class Go binding; integration is CLI orchestration (or cgo/GI). - rpm-ostree feature development is winding down in favor of bootc — a Linux adapter must track the bootc transition rather than build on rpm-ostree's growth. - Several operational specifics (GC retention defaults, delta size ratios, storage overhead constant, `--json` status, exact rollback command for current version) are **UNVERIFIED** and need hands-on confirmation/benchmarking.

§13. Recommendation

Shortlist for the Linux phase only; exclude from the Android-15-first device path. OSTree (libostree) is a strong, mature candidate as the **Linux OS-adapter backend** for file-based atomic deployments, with pinning and static deltas that map well onto Helix's known-good-baseline and bandwidth requirements, and a server model that drops cleanly into the existing Go + S3/MinIO stack. It is **not** a substitute for the locked Android native A/B strategy and must not be positioned as such.

Before any ADR commits to OSTree for Linux: (1) prototype an `archive` repo + static-delta pipeline driven by the Go control plane and **measure** delta sizes, storage overhead, and GC/pinning behavior on representative images; (2) evaluate **bootc** head-to-head, given the ecosystem shift; (3) confirm the flagged UNVERIFIED items against the pinned release. Overall confidence in the mechanism description is high; confidence in specific numeric/operational defaults is low pending benchmarking.

§14. Sources consulted

All consulted live on 2026-06-07.

- OSTree Atomic Upgrades — https://ostreedev.github.io/ostree/atomic-upgrades/
- OSTree Repository Format — https://ostreedev.github.io/ostree/repo/
- OSTree Formats (static deltas, archive/bare modes) — https://ostreedev.github.io/ostree/formats/

- `ostree admin pin` man page — https://ostreedev.github.io/ostree/man/ostree-admin-pin.html
- OSTree Bootloaders (Android aboot backend) — https://ostreedev.github.io/ostree/bootloaders/
- OSTree Introduction — https://ostreedev.github.io/ostree/introduction/
- libostree home / license — https://ostreedev.github.io/ostree/
- ostreedev/ostree GitHub (README, releases) — https://github.com/ostreedev/ostree , https://github.com/ostreedev/ostree/releases , https://github.com/ostreedev/ostree/releases/tag/v2025.7
- rpm-ostree home — https://coreos.github.io/rpm-ostree/
- coreos/rpm-ostree GitHub (README, container.md) — https://github.com/coreos/rpm-ostree
- RHEL 10: Migrating from rpm-ostree to bootc — https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/composing_installing_and_managing_rhel_for_edge_images/migrating-from-rpm-ostree-based-deployed-systems-to-bootc-based-systems
- Pinning Deployments in OSTree-Based Systems (Dusty Mabe / Project Atomic) — https://dustymabe.com/2018/05/22/pinning-deployments-in-ostree-based-systems/
- Apertis: OSTree updates and rollback — https://www.apertis.org/guides/ostree/
- Add `ostree admin rollback` (issue #1808) — https://github.com/ostreedev/ostree/issues/1808
- Rust `ostree` crate (binding reference) — https://docs.rs/crate/ostree/latest

§15. Confidence

Medium-high overall. Mechanism, deployment model, pinning, static deltas, server model, licensing, and Android-fit reasoning are well-supported by official docs consulted this session (high confidence). Specific operational numbers (GC retention defaults, storage-overhead constant, delta size ratios, exact current-version rollback command, `--json` status, exact v2025.7 release date) are flagged UNVERIFIED and lower confidence pending hands-on verification — none were invented.